

GNNs: From CNNs to Message Passing



Lecture 12 - PDF

You are a deep learning expert, focused on U.C. Berkeley's CS 182 (Deep Neural Networks) course. I want you to first provide a comprehensive guide for the first one-third of this lecture. Then I will reprompt you for more. The end goal is to work towards you giving debugging problems to me (I have to debug some sort of setup). So aim your guide towards relevant knowledge.



That's a great topic! Let's dive into **Graph Neural Networks (GNNs)**, which is the focus of this lecture. I'll guide you through the first part of the material, which sets up the crucial concepts for generalizing CNNs to arbitrary graph structures.

I'll be asking guiding questions along the way to make sure we're building a strong foundation.

1. The Core Motivation: Generalizing Structure

The lecture opens with a visual comparison (Figure 10.9 in the slides) that quickly establishes the need for structure in deep learning.

Concept	Fully Connected (a)	Weight-Shared / Local (b)
Connectivity	Every input x_i connects to every output h_j [Image 1]	Sparse/Local connections only [Image 2]



Concept	Fully Connected (a)	Weight-Shared / Local (b)
Weights	6×6 Weight Matrix with unique values [Image 3]	6×6 Weight Matrix with shared/zeroed weights [Image 4]
Insight	A standard MLP (Fully Connected) ignores the spatial relationship between x nodes, treating them as independent.	A CNN enforces weight-sharing and local connectivity to respect the spatial symmetry (like translational invariance) present in images.

This comparison is the entire philosophical jump from MLPs to CNNs. The journey to GNNs is about taking that philosophy and applying it to non-grid data (general graphs).

Nice introduction with a easy-to-understand MLP vs CNN comparison which provides good intuition. No complaints here

2. CNN Concepts that Transfer to GNNs

To design a GNN, we first recall which components of a CNN are **architecture-independent** and which are **structure-dependent**.

Concepts that Transfer 	Concepts that Don't Transfer Easily 
Weight-Sharing (Nodes do the same thing)	Pooling/Downsampling (No clear way to group arbitrary nodes)
Additional Channels/Depth	Upsampling/U-Nets
Nonlinearities	
Residual Connections	
Dropout	

This distinction is crucial for debugging: if your GNN setup is failing, it's probably *not* a problem with a residual connection, but it *could* be a bad normalization choice or an incorrect attempt at a pooling layer.

Nice visual table but would have liked a better explanation for why these concepts do not transfer easily.

3. The GNN Generalization: Message Passing

The first conceptual step in creating a GNN is viewing an **image as a graph** where the **nodes are pixels**.

In a CNN, the output feature vector h_{out} for a pixel is computed as a weighted sum of the input features $h_{in,i}$ of its 3×3 neighbors.

Above sentence is technically hallucination (not always 3x3)

The lecture generalizes this local processing into the **GNN message-passing equation** for a single node:

$$h_{out} = b + \sum_i w_i h_{in,i}$$

This equation is your GNN "convolution." Instead of 3×3 neighbors, the sum is over the node's arbitrary neighbors.

- w_i : A **weight matrix specific to the edge label** (or direction). For a directed grid graph (like the 3x3 example), this means you have one weight matrix for each direction (N, NE, E, SE, S, SW, W, NW) plus a special self-connection.
- $h_{in,i}$: The **input feature vector** from the neighboring node i .
- b : A **bias vector** (the "self-specific" matrix).

This formulation immediately allows us to generalize CNNs to **directed graphs with discrete, finite edge labels**.

This is a great starting point for understanding the architecture. The next logical step is to explore how this *convolution* concept must change for **undirected graphs**, which introduces the concept of **permutation invariance**.

Based on what we've covered, do you have a grasp of **how the GNN convolution differs from a standard MLP layer** in terms of the number and application of weights?

Appreciate the checking for understanding



go to the next one-third



That's right, let's keep going. The first part established the foundation: a GNN is essentially a generalization of convolution to a directed graph with discrete edge labels. The second part of the lecture dives into the major architectural challenges and solutions for handling **general graphs**, particularly **undirected graphs**.

4. Normalization: A Graph-Specific Challenge

One of the first things that doesn't immediately generalize from CNNs is **Batch Normalization (BN)**.

- **The Issue:** Batch Norm standardizes activations across the *batch* and *spatial dimensions* ($H \times W$). For graphs, the "spatial dimension" is the set of **Nodes**. Since graphs are arbitrary, the number of nodes per graph in a batch can vary widely. BN relies on a consistent mean/variance across the spatial dimension, which isn't guaranteed with variable graph sizes.
- **The Solution:** The easy choice is to switch to a normalization method that standardizes the activations **per embedding** (i.e., per node feature vector). This is analogous to **Layer Normalization** or **Instance Normalization** (when applied across the channels/features of a single node's embedding). This avoids the need to average statistics across potentially heterogeneous batches of graphs.

Solid explanation of why Layer Norm is better for GNNs, no significant complaints here.

5. Undirected Graphs and Permutation Invariance



The model defined by the basic message-passing formula (using a weight matrix W_i per edge label/direction) works well for directed graphs where the edge labels are fixed and finite. But for general or undirected graphs, we encounter a fundamental constraint.

- **The Constraint:** In an undirected graph, a node's neighbors simply form an unordered **set** $\{h_1, h_2, \dots, h_n\}$. The order in which the neighbors are listed or processed by the GNN cannot change the result. Therefore, the function that aggregates the neighbor information must be **permutation-invariant**.
- **The General Formulation:** This leads to the canonical GNN message-passing update for a node h_0 :

$$h_{out} = f_{W_1} (h_0, \Sigma_{i=1n} g_{W_2} (h_i))$$

- **** Σ is the Aggregation Operator:**** This is the core of the GNN. The operator Σ must be permutation-invariant. Common choices for Σ include:
 - **Sum**
 - **Average**
 - **Max/Min**

Explanation of permutation invariance and reasoning is solid but information on why sum isn't always the best choice is not touched on. And this becomes one of the debugging questions towards the end

6. Generalizing with Learned Similarity (Attention) 🧐

The GNN formulation can be made even more powerful by allowing the aggregation to be weighted by a learned **similarity** between the central node h_0 and its neighbor h_i . This leads to a more general form:

$$h_{out} = f_{W_1} (h_0, \Sigma s_{\omega_2} (h_0, h_i) g_{\omega_3} (h_i))$$

- $s_{W_2} (h_0, h_i)$: This is the **learned similarity** or **attention score** that determines how much the message from h_i contributes to the update of h_0 .

• **Connection to Transformers:** This approach is closely related to **attention mechanisms** found in Transformers. In fact, the lecture notes suggest that **Transformers can be usefully thought of as GNNs on a complete graph** (where every node is connected to every other).

Pointing this mental model is nice

Let's focus on the key constraint:

Why is it so critical that the aggregation function Σ be **permutation-invariant** for an **undirected graph**? What would go wrong if we used a function that was *not* permutation-invariant (like a simple list concatenation)?

Again appreciate the checking for understanding



End the guide with the last one-third of the slides now



Let's complete the guided overview of this lecture material. The final third focuses on the full GNN building block, the crucial role of non-linearities and skip connections, and the main tasks GNNs are used for.

7. The Full GNN Building Block (The Attention Model) 🏗️

The most general and powerful formulation of a GNN layer incorporates a learned similarity (or attention) mechanism. This is often called a **Graph Attention Network (GAT)** or similar attention-based GNN:

$$h_{out} = f_{W_1} (h_0, \Sigma s_{\omega_2} (h_0, h_i) g_{\omega_3} (h_i))$$



This equation breaks down the layer's operation into three learned functions, which are often implemented as Multi-Layer Perceptrons (MLPs) or simple linear layers:

Component	Function	Role
** $s_{W_2}(h_0, h_i)$ **	Similarity/Attention s	Learns the weight (or importance) of the message from neighbor h_i to the central node h_0 . Often uses an inner product for similarity.
** $g_{W_3}(h_i)$ **	Message Transformation g	Learns to transform the neighbor's feature vector h_i into a message vector v_i before aggregation.
** $f_{W_1}(\dots)$ **	Combination/MLP f	Learns to combine the central node's feature h_0 with the aggregated message (Σ) to produce the final output h_{out} .

This approach is highly related to **Transformer** models, which can be viewed as GNNs applied to a **complete graph** (where every node is connected to every other).

Table to display each part is helpful

8. Architectural Necessities: Non-Linearity and Residuals

When building very deep GNNs (stacking many layers), two concepts are absolutely critical:

- Non-Linearity:** Since g and f are often linear transformations (or simple MLPs with non-linearities inside), the overall operation must have an outer non-linearity (like $ReLU$ or σ) after the final combination f . Without it, stacking layers just results in a single, large linear operation, severely limiting expressivity.
- Skip/Residual Connections:** These connections, which feed the input h_0 directly to the layer's output, are a **prerequisite for avoiding**

oversmoothing.

- **Oversmoothing** is a key failure mode in deep GNNs where repeated averaging (via the Σ function) causes all node features in a connected component of the graph to converge to the same value, making them indistinguishable. Residual connections ensure the node retains some of its original, local information.

These summaries are strong but concise, which is good

Stochastic Depth Regularization

The lecture also touches upon **Stochastic Depth**, a regularization technique similar to Dropout, but applied to entire blocks (layers) in a deep network (like a deep residual GNN).

- During training, an entire layer block W_i is skipped with probability $(1 - p)$.
- If skipped, the input simply passes through the **identity map**.
- It helps with overfitting and creating functional redundancy across blocks, but **unlike standard Dropout, it requires no adjustment at test time** because of the identity map and the presence of normalizations.

Little bit strange that SDR was in the beginning of the lecture but is discussed in the last part of the guide. Model didn't really do a great job of organizing guide by the lecture sequence

9. Applications of Graph Neural Networks 🎯

GNNs are powerful because they can handle tasks at three different levels of structural complexity:

Application Level	Description	Example from Slide
Node Classification	Classifying individual nodes in a single, large graph (e.g., classifying a person or a single component of a room).	Predicting a class (e.g., Class 1, 2, or 3) from a node's embedding h_i .



Application Level	Description	Example from Slide
Graph Classification	Classifying an entire graph (e.g., deciding if a molecule is toxic or if a social network is real/fake).	Predicting a single output class for the entire input graph.
Edge Prediction	Predicting the presence, absence, or type of connection between two nodes.	Predicting if an edge exists between two nodes based on combining their embeddings (e.g., "Alice loves Bob").

These use cases are nice but lack of illustrations is disappointing

To move toward the debugging problem, let's focus on the deep GNN architecture. If you were stacking **ten** of these GNN blocks, which feature (the **nonlinearity** or the **residual connection**) is the most critical to prevent the node features from all becoming identical (oversmoothing)?



I think the answer is residual connections because they help the node features retain local information. Can you explain the correct answer and why the other answer is not the right one in depth? And then move on to the debugging problem.



That's **exactly right!** You've nailed the critical distinction. The answer is **residual connections**.

Let's break down why this is the case and how it differs from the role of non-linearity, which is also essential for deep networks.

1. Why Residual Connections Prevent Oversmoothing

- **The Problem: Oversmoothing:** In a GNN, the aggregation step (Σ) often involves **averaging** or **summing** the features from neighboring nodes. As you stack layers, this repeated averaging causes the feature vectors of connected nodes to become increasingly similar, eventually converging to almost the same value across the entire graph. This is called **oversmoothing**. If all the node embeddings are identical, the network has lost all local information and can't classify anything.
- **The Solution: Residuals:** A residual (or skip) connection feeds the input h_0 directly to the output of the layer. This forces the layer to learn a *residual mapping* instead of the complete output map.

$$h_{out} = Nonlinearity(h_0 + GNNBlockOutput)$$

By adding the original node feature h_0 back, the node **retains its identity** in a fixed proportion, preventing its features from being completely washed out by the averaging of its neighbors in the GNN block. It's a **prerequisite for avoiding oversmoothing**.

2. Why Non-Linearity is Not the Primary Solution

- **The Problem: Limited Expressivity:** If a deep neural network only uses linear layers, stacking ten layers is mathematically equivalent to just one large linear layer. It severely limits the complexity of the function the network can learn.
- **The Solution: Non-Linearity:** Functions like ReLU or Sigmoid are added after the weighted sums (or aggregations) to introduce non-linearity, allowing the network to approximate any continuous function (Universal Approximation Theorem).

Non-linearity solves the **expressivity problem**—it allows the network to be **deep**—but it does not inherently solve the **oversmoothing problem**. A non-linear function applied to the output of an *already smoothed* (averaged) feature vector will still be a smoothed, indistinguishable vector.

Solid explanations all around for this section, no significant complaints here.

Debugging Problem Setup

Now, let's put on your debugger hat.

A student is training a custom **Graph Neural Network (GNN)** on a large, undirected social network graph for a node classification task. They are using an attention-based GNN architecture similar to the general form we discussed.

- **Observation 1:** Training accuracy is stuck around 50% (random guessing for 2 classes).
- **Observation 2:** When they inspect the node feature vectors (h_{out}) coming out of the first layer, they notice the magnitude (or norm) of some vectors is *much* larger than others, sometimes orders of magnitude larger.
- **Design Choice:** The student chose **Summation** ($\Sigma = \sum$) as their permutation-invariant aggregation operator.

Given this context, what is the most likely reason why choosing **Summation** is causing instability and poor performance? And what architectural fix (discussed in the slides) would immediately stabilize the network?



One problem that I see is that in a social network, some nodes would have far more connections, so summation would lead to some norms significantly outweighing others. I think we could use some sort of normalization. Help explain everything. Also, for the next debugging question, could you provide some sort of illustration with the problem so I learn visually as well?



That's exactly right! That's a great insight into one of the main architectural challenges when generalizing a network to graph data.

The problem you identified is the instability caused by the **node degree** in unnormalized aggregation.

The Summation Instability Problem 🧠

- **The Problem:** When using **Summation** (Σ), the magnitude (or ℓ_2 -norm) of the resulting aggregated message is directly proportional to the node's degree (n).
 - A **hub node** (high degree, large n) receives a massive sum of feature vectors, leading to a huge output norm.
 - A **peripheral node** (low degree, small n) receives a small sum.
- **The Consequence:** This wide variance in activation magnitudes makes training unstable (Observation 2). The optimizer struggles because the gradient updates for high-degree nodes and low-degree nodes are attempting to correct wildly different scales of error.

The Architectural Fix: Averaging 🧠

The immediate and effective fix is to change the permutation-invariant aggregation operator (Σ) from **Summation** to **Averaging** (or **Mean**):

$$h_{out} = f_{W_1} \left(h_0, \frac{1}{n} \sum_{i=1}^n g_{W_2} (h_i) \right)$$

- **Why it works:** By dividing the sum by the number of neighbors (n), the output magnitude becomes **invariant to the node's degree**. The network can now focus on the *content* (the features) of the messages, not just the *quantity* (the number of neighbors).

This is a classic distinction: when moving from fixed-sized operations (like a 3×3 convolution, where $n = 9$) to variable-sized operations (general graphs), you must account for the variable size.

Cool question and good explanation, but there was really no context in the guide for this

Debugging Problem 2: The Missing Activation

You asked for a visual illustration, so for this next problem, let's focus on a common mistake when implementing the attention-based GNN layer.

A different student is implementing an **attention-based GNN layer** where they calculate the similarity $s(h_0, h_i)$ between the central node h_0 and its neighbors h_i .

- **Observation 1:** The network's performance is poor, and it's unable to learn any meaningful relationship between nodes.
- **Observation 2:** When they inspect the calculated attention weights for a central node, they find that the scores for all neighbors are nearly the same (e.g., 0.20, 0.19, 0.21, 0.20). The network is not assigning a high or low score to any single neighbor.
- **Design Choice:** The student calculates the un-normalized attention score e_i as a simple inner product: $e_i = (h_0)^T W h_i$. They then use this e_i directly as the final message weight s_{W_2} .

What critical non-linearity did the student forget to apply to the attention scores (e_i) that is necessary for the layer to function correctly and address Observation 2? And why is this specific non-linearity, in particular, required?

Not sure if the scores given are really distinct enough for softmax to be good here, and they also are already normalized to sum to one, which makes it an even worse example honestly

I think they forgot to apply the softmax function as this helps magnify small differences. This also normalizes the scores. Could you explain what the correct answer is and why? ^



You are absolutely correct! The student forgot to apply the **Softmax function** to the raw attention scores (e_i). 

ReLU and sigmoid were discussed in the guide but not softmax. Didn't even see softmax in the lecture slides. Not sure why there was a question on this

Softmax is the essential, missing non-linearity here, and it performs both critical roles you mentioned: **normalization** and **magnification of differences**.

The Critical Role of Softmax in GNN Attention

The attention scores in a GNN layer must represent a **probability distribution** over the neighbors, indicating how much weight each neighbor's message should have.

1. Normalization (Probabilistic Constraint):

- The raw scores $e_i = (h_0)^T \mathbf{W} h_i$ are unconstrained real numbers.
- The Softmax function converts these scores into weights α_i (the final s_{W_2} in our general formula) such that:

- $0 \leq \alpha_i \leq 1$
- $\sum_i \alpha_i = 1$
- This forces the layer to view the aggregation as a **weighted average** (or expectation-style sum), ensuring that the total influence of all messages is constant, regardless of the magnitude of the raw e_i scores.  

2. Magnification (Focusing Mechanism):

- **The Softmax formula:**

$$\alpha_i = \frac{e^{e_i}}{\sum_{j \in \mathcal{N}(0)} e^{e_j}}$$

- The use of the exponential term (e^{e_i}) **magnifies small differences** in the raw scores (e_i). A slightly higher e_i will result in a disproportionately higher α_i compared to the rest.
- **Solving Observation 2:** The student's inner product alone resulted in near-identical scores (e.g., 0.20, 0.21). Softmax pushes the larger score to a significantly higher weight (e.g., 0.35) and suppresses the smaller scores, allowing the network to **selectively focus** on the most relevant neighbors. Without this, the network simply averages all neighbor messages equally, which is why it fails to learn meaningful relationships.

Summary

Missing Element	Solves	Why it's Critical
Softmax 	Observation 2 (Identical Weights)	Focuses attention by magnifying tiny score differences, and normalizes weights into a probability distribution for stable aggregation.
 Export to Sheets		

Debugging Problem 3: The "Deep GNN" Paradox

A third student attempts to build the **Deepest GNN** in the class, stacking **20 layers** of the attention-based GNN architecture.

- **Network Design:**
 - **Normalization:** Layer Normalization (per embedding) is used. (Correct, avoids batch issues). 
 - **Aggregation:** Averaging is used. (Correct, avoids degree instability).
 - **Non-linearity & Attention:** Softmax and ReLU are used correctly.
 - **The Fault:** The student **forgot to include residual connections** (skip connections) between layers. 

between layers.

- **Observation:** The network trains for a few epochs, and then the following occurs:
 1. The gradient updates stop providing meaningful changes to the weights (i.e., **gradient information is lost**).
 2. The features of all nodes in a connected component become virtually identical (i.e., **oversmoothing**).

The Illustration

The image below conceptually illustrates the oversmoothing phenomenon that occurs without residual connections.

Your Task:

1. What is the specific, combined failure mode that results from the omission of the residual connections, as evidenced by both observations?
2. Explain the causal relationship: How does the lack of a residual connection **lead** to the loss of gradient information (Observation 1) via the process described in Observation 2?

 Sources